

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 Аналитический раздел	6
1.1 Анализ предметной области	6
1.2 Сравнительный анализ существующих решений	6
1.3 Диаграмма прецедентов разрабатываемого комплекса	7
1.4 Формализация и описание информации, подлежащей хранению разрабатываемым комплексом	8
1.5 Анализ решений для организации хранилища информации . . .	9
1.6 Анализ существующих баз данных на основе формализации данных	11
1.6.1 Дореляционные базы данных	11
1.6.2 Реляционные базы данных	11
1.6.3 Постреляционные базы данных	12
1.6.4 Вывод	12
1.7 Анализ существующих типов систем управления базами данных (СУБД) по способу доступа	12
1.7.1 Файл-серверные СУБД	13
1.7.2 Клиент-серверные СУБД	13
1.7.3 Встраиваемые СУБД	14
1.7.4 Сравнительный анализ рассмотренных СУБД	14
1.8 Выводы	15
2 Конструкторский раздел	16
2.1 Архитектура разрабатываемого программно-алгоритмического комплекса	16
2.2 Взаимодействие компонентов программно-алгоритмического комплекса	16
2.3 Ключевые структуры данных	17
2.4 Схема алгоритма монтирования устройства	18
2.5 Схема алгоритма удаления устройства из списка доверенных устройств	19

2.6	Схема алгоритма обработки узлов устройств и записей о точках монтажирования при инициализации приложения	20
2.7	Схема алгоритма проверки записи о монтажировании на предмет актуальности	22
2.8	Описание сущностей проектируемой базы данных	23
2.9	Диаграмма проектируемой базы данных	23
2.10	Описание проектируемых ограничений целостности базы данных	24
3	Технологический раздел	26
3.1	Выбор языка и среды программирования	26
3.2	Выбор средства тестирования кода	27
3.3	Обеспечение связи между клиентом и сервером	27
3.4	Выбор встраиваемой системы управления реляционными базами данных (СУБД)	28
3.5	Взаимодействие пользователя с приложением	28
3.6	Результаты тестирования	28
4	Исследовательский раздел	29
4.1	Постановка исследования	29
4.2	Технические характеристики	29
4.3	План исследования	29
4.3.1	План первой части исследования	29
4.3.2	План второй части исследования	30
4.4	Результаты исследования	30
4.4.1	Результаты первой части исследования	30
4.4.2	Результаты второй части исследования	31
	ЗАКЛЮЧЕНИЕ	33
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	36
	ПРИЛОЖЕНИЕ А	37

ВВЕДЕНИЕ

Использование внешних запоминающих устройств создаёт риск попадания на компьютер вредоносного программного обеспечения и увеличивает вероятность несанкционированного копирования информации на сторонние носители.

Цель данной работы — разработка и реализация программно-алгоритмического комплекса, снижающего риски использования внешних запоминающих устройств.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать существующие решения;
- формализовать требования к разрабатываемому комплексу;
- разработать архитектуру программно-алгоритмического комплекса;
- обосновать выбор средств программной реализации;
- разработать программное обеспечение комплекса;
- провести тестирования комплекса;
- исследовать характеристики разработанного программного обеспечения.

1 Аналитический раздел

В данном разделе анализируется предметная область, существующие решения, формализуются требования к разрабатываемому комплексу.

1.1 Анализ предметной области

Наиболее часто встречающимся интерфейсом для подключения внешних запоминающих устройств к вычислительной технике является USB (Universal Serial Bus). К таким устройствам относятся флеш-карты и жёсткие диски. Наиболее простой способ устранения рисков, связанных с использованием внешних запоминающих устройств, — программное отключение USB-портов. Однако это не всегда может быть удобно, так как порты используются и устройствами ввода-вывода, например, мышью или клавиатурой. Более рациональным решением будет не полностью отключать порты, а программно изменять поведение компьютера при подключении устройства через USB-порт.

1.2 Сравнительный анализ существующих решений

Все существующие продукты делятся на две группы — корпоративные системы предотвращения утечек и утери данных, и решения для личного пользования. Корпоративные решения имеют распределённую архитектуру, рассчитаны на управление и мониторинг множества рабочих станций, и являются избыточными для персонального компьютера. В анализе участвуют только программы для личного использования.

Для анализа были выбраны следующие программные комплексы:

1. USBGuard [1];
2. USBFilter [2];
3. USBWall [3];
4. USBShield [4];
5. Ratool [5];
6. GiliSoft USB Lock [6];
7. USB Block [7].

В таблице 1.1 представлены результаты анализа существующих решений.

Таблица 1.1 – Результат анализа существующих решений

Критерий	Номер решения						
	1	2	3	4	5	6	7
поддержка ОС (операционной системы) Linux Ubuntu	+	+	+	-	-	-	-
возможность создания списка доверенных устройств	+	+	+	+	-	+	+
возможность установки даты истечения периода доверия к устройству	-	-	-	-	-	-	-
наличие графического интерфейса	-	-	-	+	+	+	+
наличие обновлений и поддержки	+	-	-	+	+	+	+

Таким образом, представленные решения не соответствуют в полной мере поставленным критериям.

1.3 Диаграмма прецедентов разрабатываемого комплекса

На рисунке 1.1 представлена диаграмма прецедентов разрабатываемого комплекса.



Рисунок 1.1 – Диаграмма прецедентов комплекса

1.4 Формализация и описание информации, подлежащей хранению разрабатываемым комплексом

Формальное описание устройства может быть представлено следующими атрибутами:

1. идентификатор производителя;
2. идентификатор устройства;
3. уникальный серийный номер устройства;
4. наименование производителя;
5. наименование устройства.

Доверенное устройство характеризуется формальным описанием устройства и датой истечения доверия.

Запись о монтаже устройства описывается следующей информацией:

1. формальное описание устройства;
2. узел устройства;
3. путь монтирования;
4. режим монтирования (только для чтения или полный доступ).

1.5 Анализ решений для организации хранилища информации

Существует несколько основных способов организации хранилища, к которым относятся базы данных и плоские файлы.

В таблице 1.2 представлены результаты анализа хранилищ данных.

Таблица 1.2 – Результат анализа решений для хранения списка доверенных устройств

Критерий \ Решение	Плоские файлы	Базы данных
наличие встроенных механизмов поддержания целостности данных (ограничения, уникальность, проверки)	нет	да
устойчивость хранилища к сбоям и некорректному завершению работы (восстановление согласованного состояния)	нет	да
отсутствие дополнительных зависимостей у прикладного приложения	да	нет
отсутствие возможности несанкционированного ручного просмотра и редактирования данных без специализированных инструментов	нет	да
возможность выполнения выборки и фильтрации данных без дополнительной реализации алгоритмов поиска	нет	да
отсутствие необходимости инициализации и настройки хранилища перед началом работы	да	нет
возможность расширения структуры данных и добавления новых требований без переработки существующей логики хранения	нет	да

На основе таблицы анализа для разрабатываемого комплекса предпочтительнее использовать базу данных.

1.6 Анализ существующих баз данных на основе формализации данных

Базы данных подразделяются на три основных типа по модели данных:

1. дореляционные;
2. реляционные;
3. постреляционные.

1.6.1 Дореляционные базы данных

Дореляционные базы данных [8] хранят записи в виде связей предок/потомок. Основными недостатками дореляционных баз данных являются избыточность данных и сложность выполнения запросов. Изменение схемы базы данных крайне ограничена. Целостность данных обеспечивается за счёт жёсткой структуры связей. Пример хранения записей в дореляционных базах представлен на рисунке 1.2.

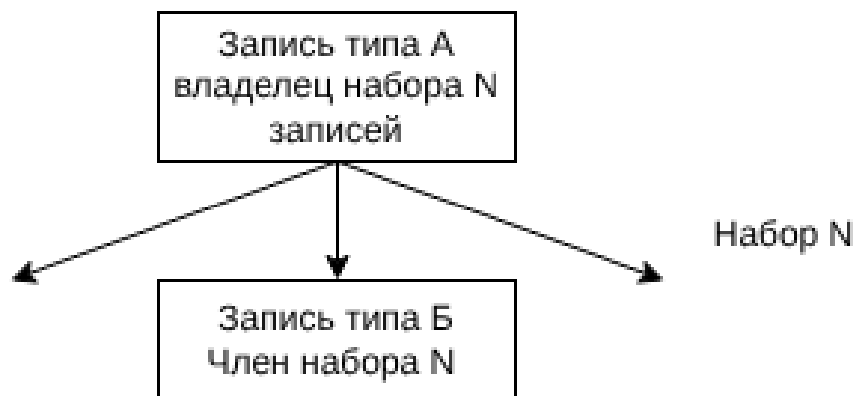


Рисунок 1.2 – Пример хранения записей в дореляционных базах данных

1.6.2 Реляционные базы данных

В реляционных базах данных данные хранятся в виде двумерных таблиц. Связь между ними обеспечивается общими столбцами, такими как идентификаторы. Табличная архитектура подходит для хранения структурированной информации. Целостность и непротиворечивость данных соблюдается при помощи ограничений. Пример хранения записей в реляционных базах представлен на рисунке 1.3.

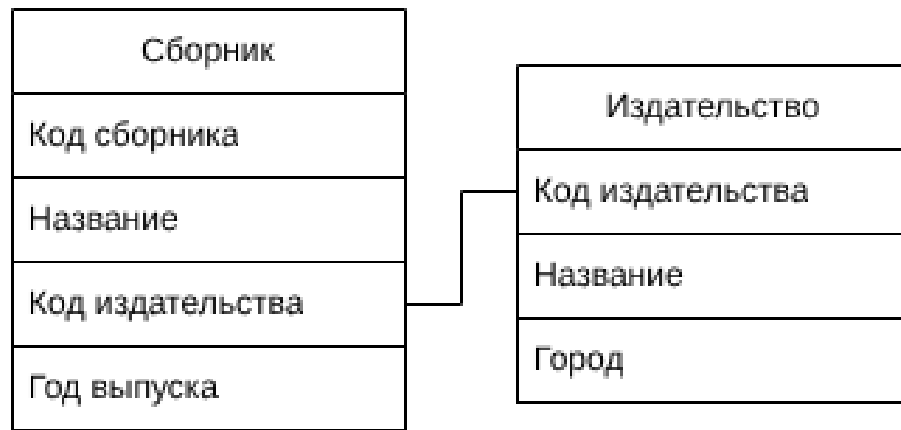


Рисунок 1.3 – Пример хранения записей в реляционных базах данных

1.6.3 Постреляционные базы данных

Постреляционные базы данных подходят для хранения неструктурированных данных, поддерживая логические связи между ними. Недостатком является сложность обеспечения целостности данных. Отличаются хорошей горизонтальной масштабируемостью, сложность запросов варьируется в зависимости от конкретной реализации.

1.6.4 Вывод

Для реализации поставленной задачи была выбрана реляционная модель, так как она лучше всего подходит для хранения структурированной информации, обеспечивая баланс между целостностью данных, простотой запросов и возможностями масштабирования.

1.7 Анализ существующих типов систем управления базами данных (СУБД) по способу доступа

СУБД по способу доступа к базе данных подразделяется на три основных вида [9]:

1. файл-серверные;
2. клиент-серверные;
3. встраиваемые.

1.7.1 Файл-серверные СУБД

При работе с файл-серверной СУБД обработка данных происходит на рабочем месте, а сервер применяется только как отдельный накопитель. Каждый пользователь сам использует информацию и вносит изменения в файлы данных и в индексные файлы. На рисунке 1.4 представлена общая схема работы файл-серверных СУБД.

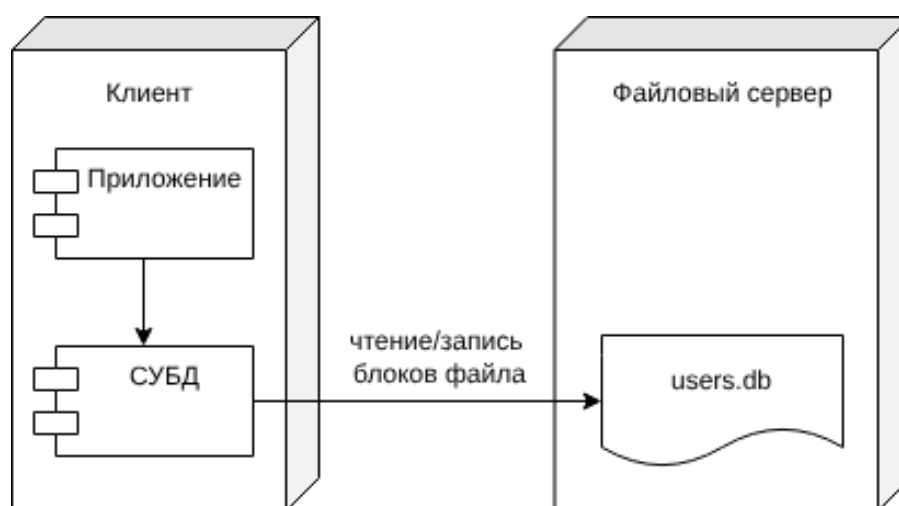


Рисунок 1.4 – Схема работы файл-серверных СУБД

1.7.2 Клиент-серверные СУБД

Клиент-серверная СУБД позволяет обмениваться клиенту и серверу минимально необходимыми объёмами информации. Клиент может выполнять функции предварительной обработки перед передачей информации серверу, но в основном его функции заключаются в организации доступа пользователя к серверу. В большинстве случаев клиент-серверная СУБД гораздо менее требовательна к пропускной способности компьютерной сети, чем файл-серверная СУБД, особенно при выполнении операции поиска в базе данных по заданным пользователем параметрам, т.к. для поиска нет необходимости получать на клиент весь массив данных: клиент передаёт параметры запроса серверу, а сервер производит поиск по полученному запросу в локальной базе данных. Результат выполнения запроса возвращается клиенту, который обеспечивает отображение результата пользователю. На рисунке 1.5 представлена общая схема работы клиент-серверных СУБД.

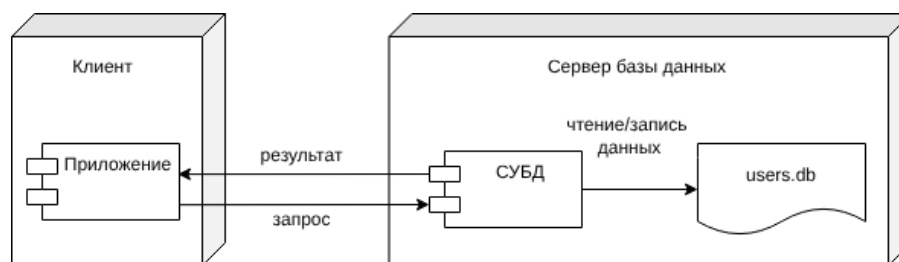


Рисунок 1.5 – Схема работы клиент-серверных СУБД

1.7.3 Встраиваемые СУБД

Встраиваемая система управления базой данных — это система, которая может быть связана с клиентским приложением таким образом, чтобы приложение и СУБД работали в едином адресном пространстве. Вместе со встроенной базой данных приложение может быть развернуто как единая программа. Благодаря связыванию приложения с базой данных, прикладная система выигрывает от снижения общей сложности и уменьшения затрат на администрирование. На рисунке 1.6 представлена общая схема работы встраиваемых СУБД.

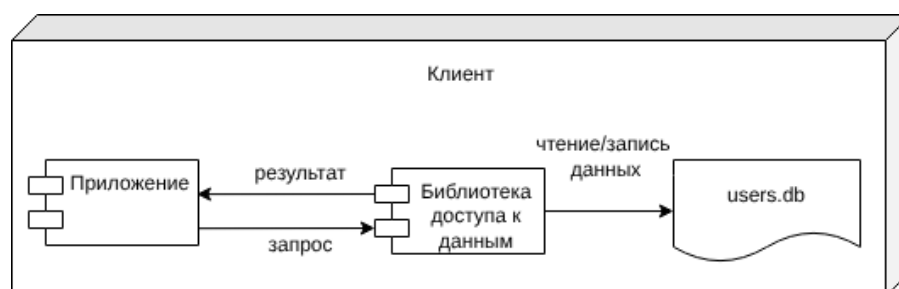


Рисунок 1.6 – Схема работы встраиваемых СУБД

1.7.4 Сравнительный анализ рассмотренных СУБД

В таблице 1.3 представлены результаты сравнения рассмотренных СУБД.

Таблица 1.3 – Результаты сравнения СУБД по способу доступа

Критерий \ Тип СУБД	Файл-серверная	Клиент-серверная	Встраиваемая
Отсутствие избыточности для локального однопользовательского приложения	да	нет	да
Отсутствие необходимости сетевого взаимодействия	нет	нет	да
Отсутствие необходимости в отдельном серверном процессе	да	нет	да

Таким образом, на основании сравнения СУБД, для разрабатываемого программно-алгоритмического комплекса наиболее подходящей будет встраиваемая СУБД.

1.8 Выводы

В данном разделе была проанализирована предметная область, проведено сравнение существующих решений по сформулированным критериям. Программно-алгоритмический комплекс был формализован с использованием диаграммы прецедентов, описаны представления ключевых структур данных, проанализированы способы организации хранилища, выбраны наиболее подходящие типы базы данных и СУБД.

2 Конструкторский раздел

В данном разделе представлена архитектура программно-алгоритмического комплекса, описаны его компоненты, их взаимодействие и ключевые структуры данных, приведены реализуемые схемы алгоритмов, описаны сущности и ограничения целостности проектируемой базы данных.

2.1 Архитектура разрабатываемого программно-алгоритмического комплекса

Архитектура комплекса будет представлена моделью клиент-сервер, где клиент — приложение, позволяющее пользователю управлять списком доверенных устройств, а сервер — приложение, отвечающее непосредственно за монтирование устройств, хранение записей о монтировании, работе с базой данных. Схематично данная модель представлена на рисунке 2.1.

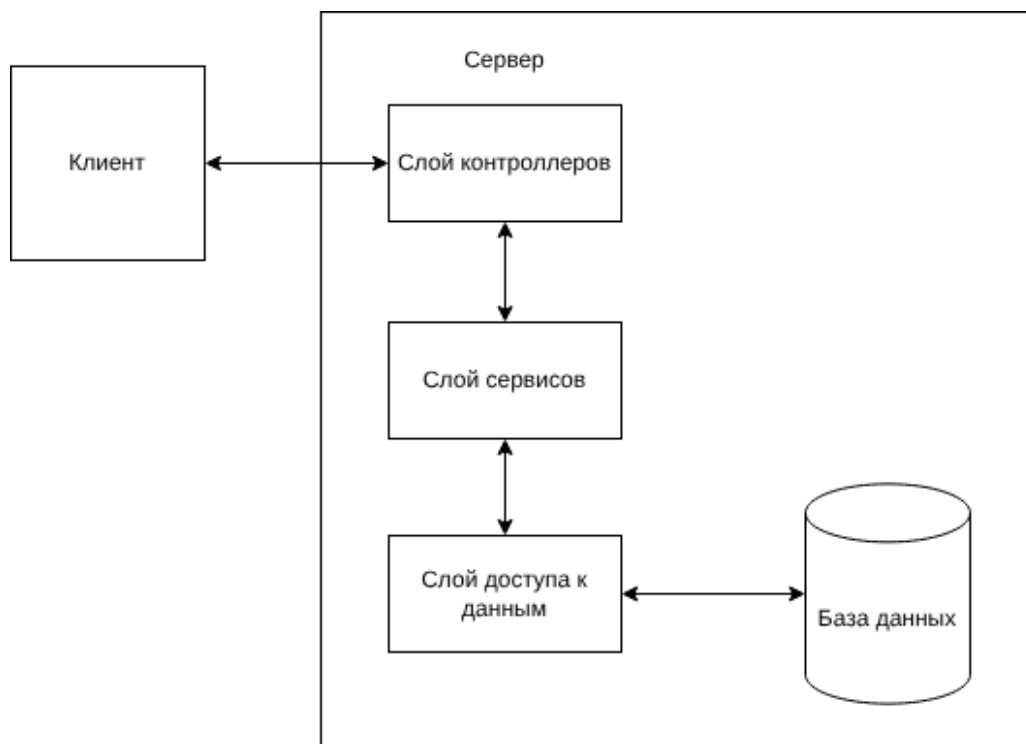


Рисунок 2.1 – Модель клиент-сервер

2.2 Взаимодействие компонентов программно-алгоритмического комплекса

Взаимодействие клиента и сервера может быть описано с помощью спецификации OpenAPI [10]. Использование OpenAPI позволяет формализовать

структуру запросов и ответов, определить параметры запросов и форматы передаваемых данных.

На рисунке 2.2 представлено описание программного интерфейса приложения, включающее маршруты взаимодействия клиента и сервера.

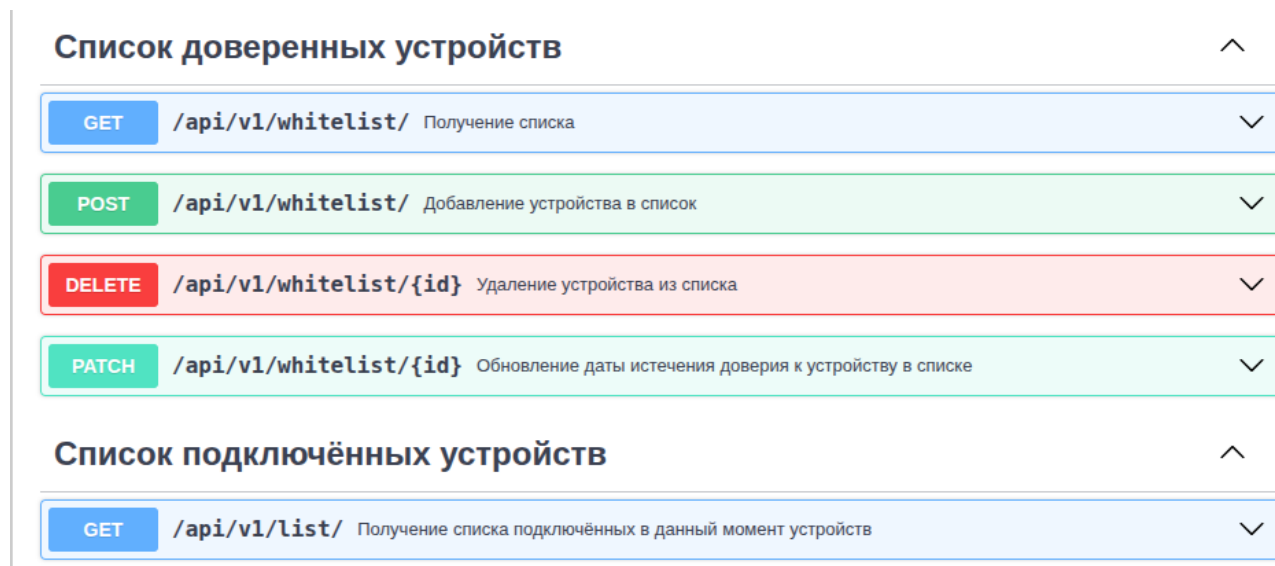


Рисунок 2.2 – Описание программного интерфейса приложения

2.3 Ключевые структуры данных

Серверная часть использует несколько структур данных для отправки ответов клиенту и использования во внутренних методах.

- DeviceInfo — структура формального описания USB-устройства:
 1. vendorId — идентификатор производителя;
 2. productId — идентификатор устройства;
 3. serial — серийный номер;
 4. vendorName — название производителя;
 5. productName — название продукта.
- Device — структура описания доверенного устройства, используется для передча на клиент при запросе списка доверенных устройств:
 1. id — первичный ключ из базы данных;
 2. info — формальное описание DeviceInfo;

3. `validTo` — дата истечения доверия.
- `EventType` — перечисляемый тип, используемый для обозначения физического манипулирования с накопителем, где `INSERT(0)` — подключение устройства, `REMOVE(1)` — извлечение устройства;
 - `DeviceEvent` — структура, описывающая физическое событие с накопителем. Имеет поля `type` типа `EventType`, и `devNode` — узел устройства;
 - `MODE` — перечисляемый тип, используемый для обозначения режима монтирования, где `RO(0)` — режим только для чтения, а `RW(1)` — режим полного доступа;
 - `MountRecord` — структура описания события монтирования, используется для передачи на клиент при запросе подключённых в данный момент устройств:
 1. `id` — первичный ключ устройства из базы данных(при наличии);
 2. `devNode` — узел устройства;
 3. `mountPoint` — путь монтирования;
 4. `info` — формальное описание `DeviceInfo`;
 5. `mode` — режим монтирования.

2.4 Схема алгоритма монтирования устройства

На рисунке 2.3 представлена схема алгоритма монтирования устройства при его подключении к компьютеру.

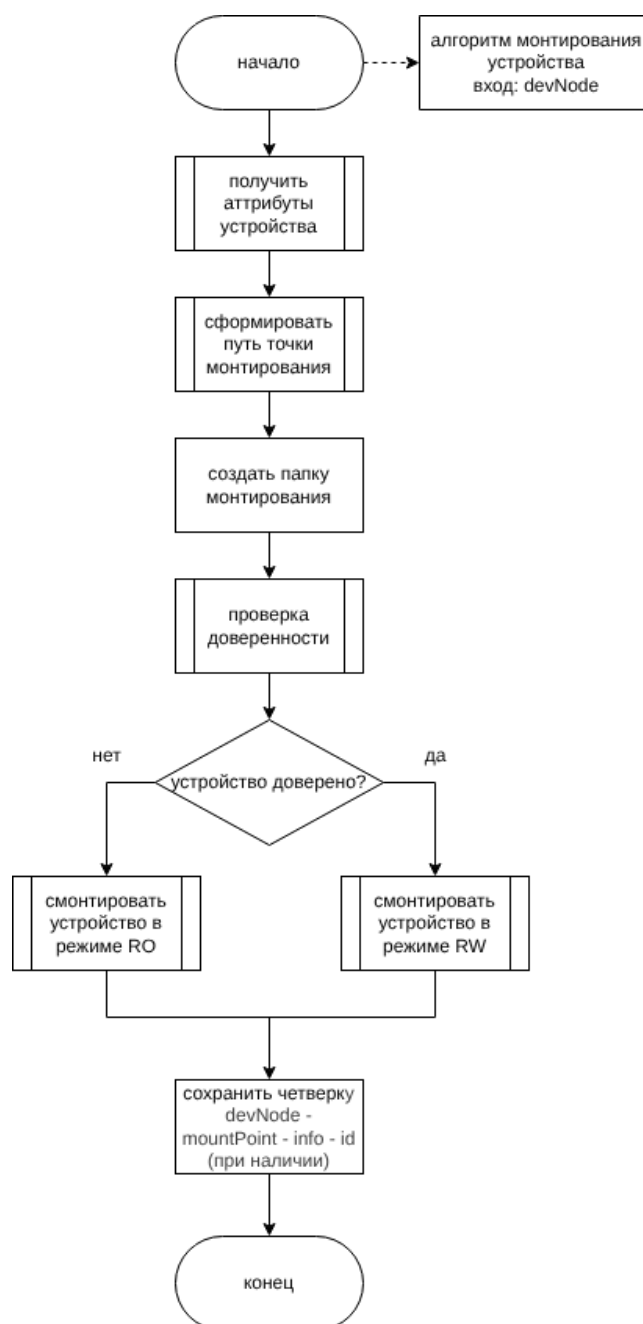


Рисунок 2.3 – Схема алгоритма монтирования

2.5 Схема алгоритма удаления устройства из списка доверенных устройств

На рисунке 2.4 представлена схема алгоритма удаления устройства из списка доверенных устройств.

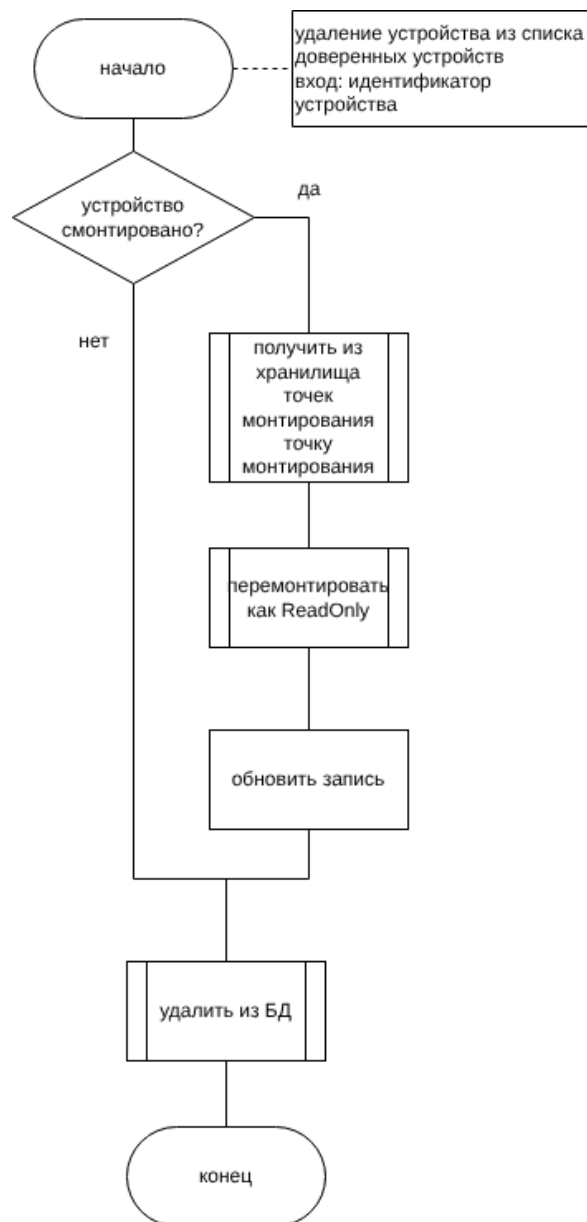


Рисунок 2.4 – Схема алгоритма удаления устройства

2.6 Схема алгоритма обработки узлов устройств и записей о точках монтирования при инициализации приложения

На рисунке 2.5 представлена схема алгоритма обработки узлов устройств и записей о точках монтирования при инициализации приложения.

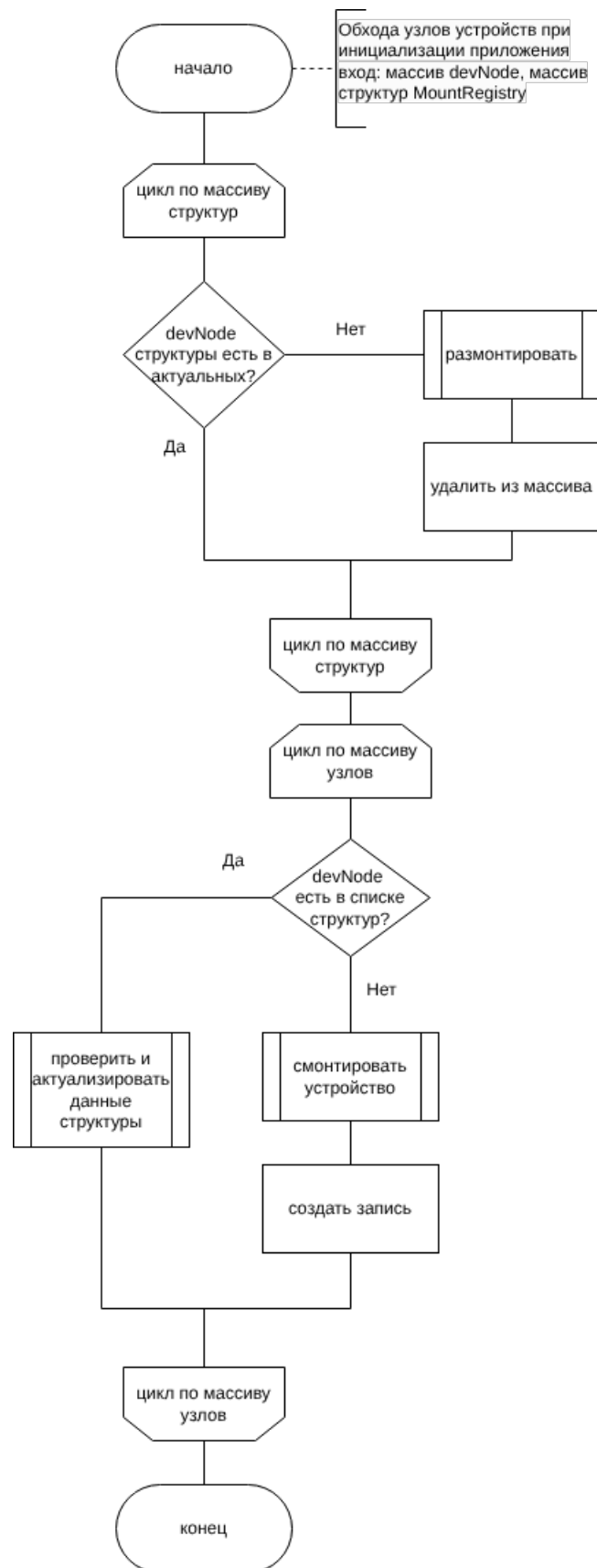


Рисунок 2.5 – Схема алгоритма обработки узлов устройств и записей о точках монтирования при инициализации приложения

2.7 Схема алгоритма проверки записи о монтировании на предмет актуальности

На рисунке 2.6 представлена схема алгоритма проверки записи о монтировании на предмет актуальности.

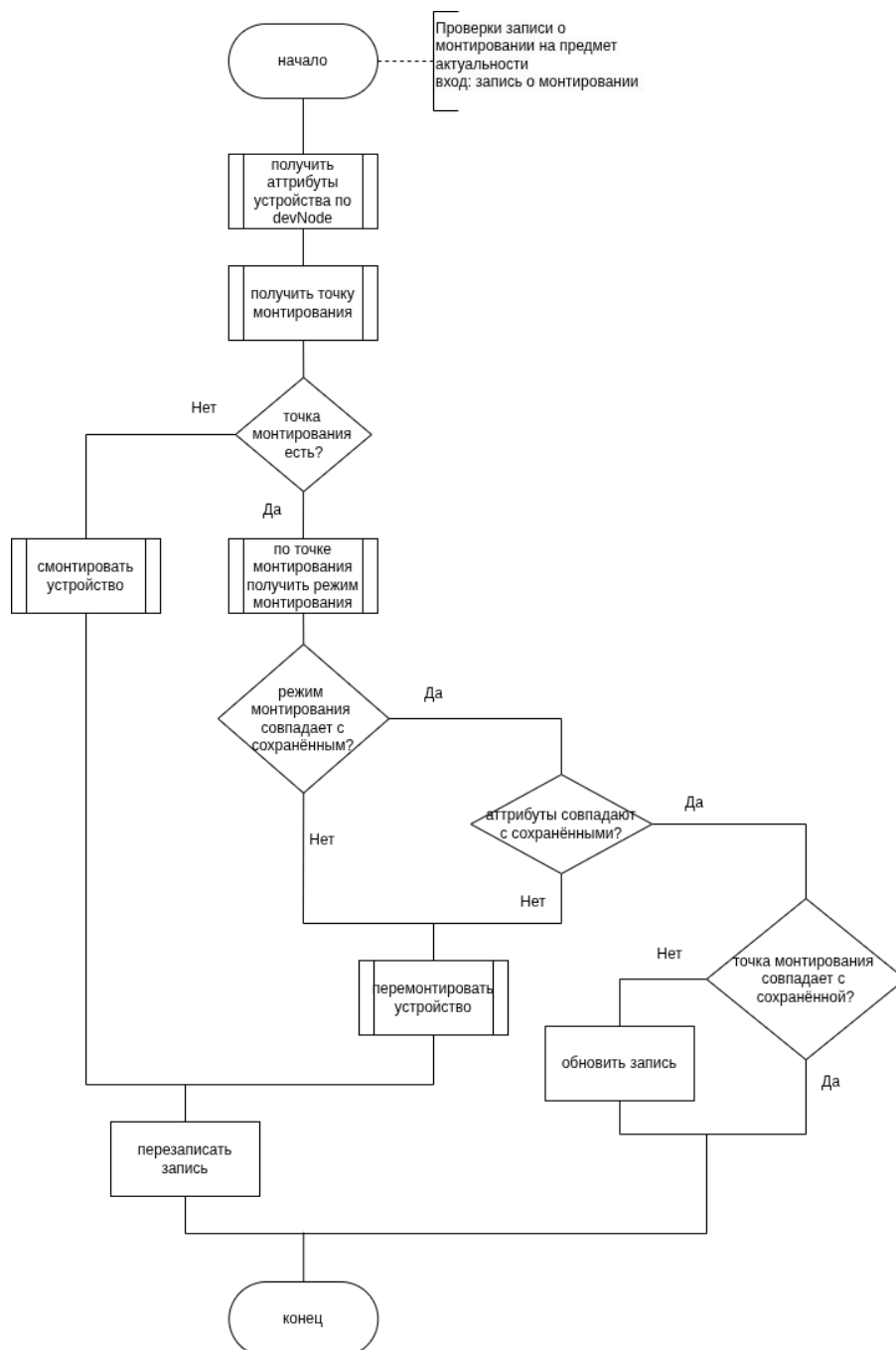


Рисунок 2.6 – Схема алгоритма проверки записи о монтировании на предмет актуальности

2.8 Описание сущностей проектируемой базы данных

База данных содержит следующие таблицы:

- DeviceInfo — описание устройства:
 1. id — первичный ключ;
 2. vendorId — идентификатор производителя;
 3. productId — идентификатор продукта;
 4. serial — серийный номер;
 5. productName — название производителя;
 6. vendorName — название устройства.
- Device — описание записи в списке доверенных устройств:
 1. id — первичный ключ;
 2. deviceInfoId — идентификатор описания устройства;
 3. validTo — дата окончания периода доверия.
- MountRecord — описание события монтирования:
 1. id — первичный ключ;
 2. deviceInfoId — идентификатор описания устройства;
 3. devNode — узел устройства;
 4. mountPoint — точка монтирования;
 5. mode — режим монтирования.

2.9 Диаграмма проектируемой базы данных

На рисунке 2.7 представлена диаграмма проектируемой базы данных.

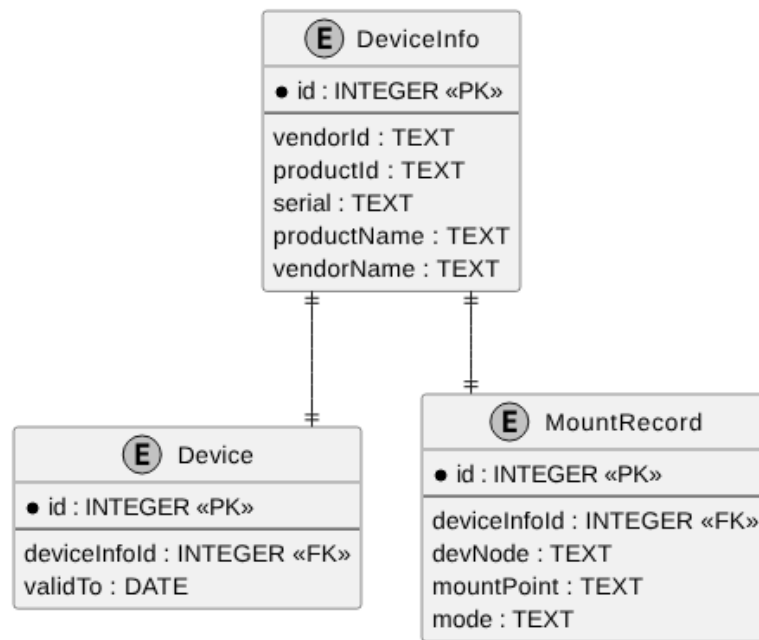


Рисунок 2.7 – Диаграмма базы данных

Между таблицами базы данных образуется связь один к одному.

2.10 Описание проектируемых ограничений целостности базы данных

На значения полей базы данных накладываются следующие ограничения целостности:

— DeviceInfo:

1. поля `vendorId` и `productId` должны содержать только цифры или строчные или заглавные латинские буквы, их длина должна равняться 4-ём символам, они не могут быть пустыми;
2. поле `serial` не может быть пустым;
3. комбинация полей `vendorId`, `productId`, `serial` должна быть уникальной.

— MountRecord:

1. поле `deviceInfoId` является внешним ключом, ссылающимся на таблицу `DeviceInfo`, должно быть непустым и уникальным;
2. `devNode` должно быть непустым и уникальным;

3. `mountPoint` должно быть непустым и уникальным;
 4. `mode` должно быть непустым и принимать только значения «RO» и «RW».
- Device: поле `deviceInfoId` является внешним ключом, ссылающимся на таблицу `DeviceInfo`, должно быть непустым и уникальным, а `validTo` должно содержать дату в формате «ГГГГ-ММ-ДД».

3 Технологический раздел

3.1 Выбор языка и среды программирования

Для разработки серверной части программно-алгоритмического комплекса был выбран язык программирования C++ [11], что обусловлено следующими факторами:

1. встроенные возможности объектно-ориентированного программирования, позволяющие создавать легко расширяемые приложения;
2. совместимость с языком C, возможность прямого вызова функций монтирования устройств;
3. поддержка многопоточности, необходимой для параллельной обработки событий;
4. стандартная библиотека предоставляет множество контейнеров, упрощающих разработку.

Для разработки клиентской части был выбран язык TypeScript [12] и фреймворк Vue.js [13]. Использование TypeScript обеспечивает статическую типизацию, что позволяет уменьшить количество ошибок на этапе разработки и повысить удобство сопровождения кода. Кроме того, язык предоставляет современные средства разработки веб-приложений и хорошо интегрируется с большинством популярных frontend-фреймворков.

Выбор Vue.js обусловлен компонентным подходом к разработке пользовательского интерфейса, что упрощает повторное использование элементов приложения и повышает удобство сопровождения проекта. Дополнительно фреймворк предоставляет встроенные средства реактивного обновления интерфейса, позволяющие эффективно отображать изменения состояния приложения в режиме реального времени.

В качестве среды разработки был выбран Visual Studio Code [14] — кроссплатформенная среда, функциональность которой может быть увеличена с использованием дополнительных расширений.

3.2 Выбор средства тестирования кода

Для тестирования серверной части приложения был выбран фреймворк GoogleTests [15]. Данный продукт позволяет создавать unit-тесты, e2e-тесты и интеграционные тесты. Разработан специально для C++, и использует стандартный его синтаксис. Позволяет реализовывать Mock-объекты для изоляции зависимостей при тестировании отдельных компонентов системы [16]. Также фреймворк поддерживает параметризованные тесты, что позволяет проверять поведение методов на различных наборах входных данных без дублирования кода тестов.

Для тестирования клиентской части приложения была выбрана библиотека Vitest [17]. Данная библиотека предназначена для тестирования JavaScript- и TypeScript-приложений, а также является нативной для фреймворка Vue.js. Vitest поддерживает создание unit- и компонентных тестов, предоставляет встроенные средства мокирования функций и модулей. Дополнительно библиотека поддерживает параллельное выполнение тестов, что сокращает время тестирования клиентской части приложения.

3.3 Обеспечение связи между клиентом и сервером

Для связи клиента и сервера используется протокол HTTP (HyperText Transfer Protocol) [18] и механизм WebSocket [19].

Протокол HTTP является протоколом прикладного уровня модели OSI (Open Systems Interconnection) и предназначен для обмена данными между клиентом и сервером по схеме «запрос-ответ». Клиент формирует HTTP-запрос, сервер обрабатывает его и возвращает ответ, содержащий необходимые данные. Протокол широко применяется при передаче гипертекстовых документов и поддерживается большинством современных программных и аппаратных средств. В разрабатываемой программе используется, например, для получения списка подключённых устройств на клиенте.

Протокол WebSocket используется для организации постоянного двустороннего соединения между клиентом и сервером в режиме реального времени. В отличие от HTTP, WebSocket обеспечивает асинхронный обмен сообщениями и уменьшает накладные расходы на передачу данных за счёт отсутствия необходимости повторного создания соединения для каждого запроса.

Установление WebSocket-соединения начинается с HTTP-запроса, после чего взаимодействие продолжается по отдельному протоколу WebSocket. В разрабатываемой программе используется для передачи на клиент информации о подключении и извлечении устройств.

3.4 Выбор встраиваемой системы управления реляционными базами данных (СУБД)

Для сравнения были выбраны следующие известные системы:

1. SQLite [20];
2. DuckDB [21];
3. H2 Database [22];
4. Apache Derby [23].

Результаты анализа существующих решений приведены в таблице 3.1.

Таблица 3.1 – Результат анализа существующих решений

Критерий	Номер решения			
	1	2	3	4
Поддержка Linux	да	да	да	да
Поддержка C++	да	да	нет	нет
Бесплатное распространение	да	да	да	да
Поддержка внешних ключей	да	да	да	да
Поддержка ограничений	да	да	да	да
Наличие обновлений и поддержки	да	да	да	нет
Отсутствие внешних зависимостей	да	да	нет	нет
Размер библиотеки(Мегабайты)	1.5	70	2.5	6.1

На основе анализа, для разрабатываемого комплекса была выбрана SQLite.

3.5 Взаимодействие пользователя с приложением

3.6 Результаты тестирования

4 Исследовательский раздел

В данном разделе проводится исследование временных характеристик разработанного программного обеспечения.

4.1 Постановка исследования

Предметом исследования является время выполнения алгоритма обработки узлов устройств и записей о точках монтирования при инициализации приложения при различных начальных условиях. В первой части исследования меняется количество записей о точках монтировании, а во второй части меняется количество актуальных записей о точках монтирования.

4.2 Технические характеристики

Исследование проводилось на виртуальной машине с операционной системой Linux Ubuntu Server. Для создания виртуальной машины использовался программный механизм KVM (Kernel-based Virtual Machine) [24].

Конфигурация виртуальной машины:

- количество процессоров — 4;
- объём оперативной памяти — 6 гигабайт;
- объём виртуального диска — 20 гигабайт.

Хостовая машина работает на процессоре 11th Gen Intel(R) Core(TM) i7-11370H @ 3.30 ГГц [25].

4.3 План исследования

Для проведения исследования с помощью модуля ядра `dummy_hcd` [26] было создано виртуальное устройство USB с восьмью LUN (Logical Unit Number).

4.3.1 План первой части исследования

Количество исходных записей о монтировании N изменяется от 0 до 8. Для каждого состояния проводится 50 замеров времени и их дальнейшее

усреднение. Перед каждым замером очищается хранилище записей о монтировании, все устройства размонтируются, N устройств из 8 монтируется, записи о них сохраняются. Количество виртуальных устройств неизменно.

4.3.2 План второй части исследования

Количество исходных записей о монтировании и количество виртуальных устройств неизменно и равно 8. Перед каждым замером очищается хранилище записей о монтировании, все устройства перемонтируются, часть записей сохраняется в достоверном виде, а N записей записи сохраняются в изменённом формате — режим монтирования меняется на противоположный, где N изменяется от 0 до 8.

4.4 Результаты исследования

В результате проведённого исследования были получены следующие результаты.

4.4.1 Результаты первой части исследования

В результате был получен график зависимости времени выполнения алгоритма обработки узлов устройств и записей о точках монтирования от количества исходных записей о точках монтирования, представленный на рисунке 4.1.



Рисунок 4.1 – Результат первой части исследования

Время выполнения алгоритма практически линейно зависит от количества записей. При наличии всех записей о монтировании время выполнения уменьшается более чем в 10 раз относительно состояния отсутствия записей.

4.4.2 Результаты второй части исследования

В результате был получен график зависимости времени выполнения алгоритма обработки узлов устройств и записей о точках монтирования от количества недостоверных записей о монтировании, представленный на рисунке 4.2.

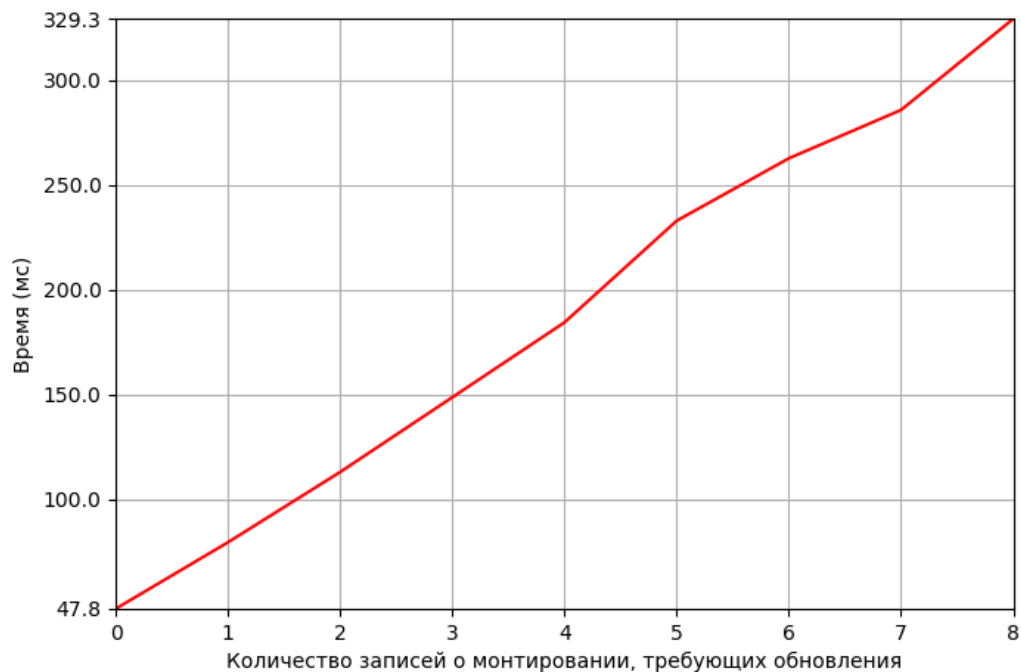


Рисунок 4.2 – Результат второй части исследования

Время выполнения алгоритма растёт при увеличении количества недо-
стоверных записей, так как при обнаружении несоответствия фактического
режима монтирования и документированного режима происходит перемон-
тирование и перезапись записи о монтировании. Наблюдается семикратное
увеличение времени выполнения при недостоверности всех 8 записей относи-
тельно полностью правильного набора.

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. USBGuard; [Электронный ресурс]. — Режим доступа: <https://usbguard.github.io/> (дата обращения: 01.03.2026).
2. Making USB Great Again with USBFILTER — a USB layer firewall in the Linux kernel; [Электронный ресурс]. — Режим доступа: <https://davejingtian.org/2016/08/04/making-usb-great-again-with-usbfilter-a-usb-layer-firewall-in-the-linux-kernel/> (дата обращения: 01.03.2026).
3. USBWall Source Code; [Электронный ресурс]. — Режим доступа: <https://github.com/usbwall/usbwall?ysclid=mndivt3urh835120502> (дата обращения: 01.03.2026).
4. USBShield Source Code; [Электронный ресурс]. — Режим доступа: <https://github.com/USBShield/USBShield?ysclid=mndiyaq4ah326619085> (дата обращения: 01.03.2026).
5. Sordum — Ratool v1.4 (Removable Access tool); [Электронный ресурс]. — Режим доступа: <https://www.sordum.org/8104/ratool-v1-4-removable-access-tool/> (дата обращения: 01.03.2026).
6. Gilisoft USB Lock — Control USB ports, removable devices, and trusted-device access to reduce data leakage on Windows; [Электронный ресурс]. — Режим доступа: <https://www.gilisoft.com/product-usb-lock.htm> (дата обращения: 01.03.2026).
7. NewSoftwares — USBBlock. Restrict Access of Portable Drives; [Электронный ресурс]. — Режим доступа: <https://www.newsoftwares.net/usb-block/> (дата обращения: 01.03.2026).
8. Модели баз данных: учебное пособие / О.Е. Аврунев, В.М. Стасышин. — Новосибирск: Изд-во НГТУ, 2018. — 124 с.
9. Создание базы данных для разработки облачной платформы хранения и редактирования 3D моделей конфет / В. Г. Благовещенский, А. Е. Краснов, И. Г. Благовещенский [и др.] // Интеллектуальные автоматизированные управляющие системы в биотехнологических процессах : сборник докладов всероссийской научно-практической конференции,

- Москва, 29 марта 2023 года. – Москва: РОССИЙСКИЙ БИОТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ; ЗАО «Университетская книга», 2023. – С. 85-96. – EDN NAJYYS.
10. OpenAPI Initiative — The OpenAPI Initiative; [Электронный ресурс]. — Режим доступа: <https://www.openapis.org/> (дата обращения: 01.03.2026).
 11. Standard C++; [Электронный ресурс]. — Режим доступа: <https://isocpp.org/> (дата обращения: 01.03.2026).
 12. TypeScript — JavaScript With Syntax For Types; [Электронный ресурс]. — Режим доступа: <https://www.typescriptlang.org/> (дата обращения: 01.03.2026).
 13. Vue.js — The Progressive JavaScript Framework | Vue.js; [Электронный ресурс]. — Режим доступа: <https://vuejs.org/> (дата обращения: 01.03.2026).
 14. Visual Studio Code — The open source AI code editor; [Электронный ресурс]. — Режим доступа: <https://code.visualstudio.com/> (дата обращения: 01.03.2026).
 15. GoogleTest User's Guide; [Электронный ресурс]. — Режим доступа: <https://google.github.io/googletest/> (дата обращения: 01.03.2026).
 16. Тюменцев, Е. А. Автоматизированное тестирование сложности алгоритмов с помощью Mock-объектов / Е. А. Тюменцев // Математические структуры и моделирование. — 2013. — № 1(27). — С. 82-88. — EDN QLJADF.
 17. Vitest — Next Generation testing framework; [Электронный ресурс]. — Режим доступа: <https://vitest.dev/> (дата обращения: 01.03.2026).
 18. Борисов П. Е. HTTP-протокол прикладного уровня (обзор) // Научный аспект. — 2024. — Т. 22. — №. 5. — С. 2937-2943.
 19. Хабаров, С. П. Взаимодействия узлов сети по протоколу WebSocket / С. П. Хабаров // Информационные системы и технологии: теория и практика : Сборник научных трудов научно-технической конференции института леса и природопользования, Санкт-Петербург, 01 февраля 2017 года. Том Выпуск 9. — Санкт-Петербург: Санкт-Петербургский

- государственный лесотехнический университет им. С.М. Кирова, 2017. — С. 104-115. — EDN YRLWFS.
20. SQLite Home Page; [Электронный ресурс]. — Режим доступа: <https://www.sqlite.org/> (дата обращения: 01.03.2026).
 21. DuckDB — An in-process SQL OLAP database management system; [Электронный ресурс]. — Режим доступа: <https://duckdb.org/> (дата обращения: 01.03.2026).
 22. H2 Database Engine — Welcome to H2, the Java SQL database; [Электронный ресурс]. — Режим доступа: <https://h2database.com/html/main.html> (дата обращения: 01.03.2026).
 23. Apache Derby; [Электронный ресурс]. — Режим доступа: <https://db.apache.org/derby/> (дата обращения: 01.03.2026).
 24. KVM — Main Page; [Электронный ресурс]. — Режим доступа: https://linux-kvm.org/page/Main_Page (дата обращения: 01.03.2026).
 25. Intel Core i7-11370H Specs — TechPowerUp CPU Database; [Электронный ресурс]. — Режим доступа: <https://www.techpowerup.com/cpu-specs/core-i7-11370h.c2833> (дата обращения: 01.03.2026).
 26. Linux source code (v7.0.8) — Bootlin Elixir Cross Referencer; [Электронный ресурс]. — Режим доступа: https://elixir.bootlin.com/linux/v6.7-rc3/source/drivers/usb/gadget/udc/dummy_hcd.c#L82 (дата обращения: 01.03.2026).

ПРИЛОЖЕНИЕ А